

Toxic Comment Detection with Scikit-learn

Project Report - Advanced Python for NLP

Multi-label classification with TF-IDF and Logistic Regression

Dataset: Jigsaw Toxic Comment Classification Challenge

Implementation: Python, pandas, scikit-learn, joblib, Gradio

Report based on the final scikit-learn notebook and recorded outputs

Mohamed Haykel Herrouz

Uni-Kennung : vej40wig

Abstract

This project implements a multi-label toxic comment classifier using the Jigsaw Toxic Comment Classification Challenge dataset. The system represents English comments with TF-IDF features using unigrams and bigrams and trains six binary Logistic Regression classifiers through scikit-learn's One-vs-Rest strategy. The dataset contains 159,571 comments and six toxicity labels: toxic, severe_toxic, obscene, threat, insult, and identity_hate. A reproducible 70/20/10 train-validation-test split is used. On the final test split, the model achieves micro precision 0.9026, micro recall 0.5118, and micro F1 0.6532. The results show strong precision but substantially lower recall, especially for rare classes such as threat and identity_hate. Qualitative tests also reveal a clear limitation with negation: phrases such as "you are not stupid" remain classified as toxic with high confidence. These results show that a comparatively simple scikit-learn pipeline can provide a strong and fast baseline while also exposing limitations caused by class imbalance and shallow lexical representations.

1. Project Goal and Research Questions

The main goal is to build a complete, reproducible NLP classification pipeline using techniques covered in the Advanced Python for NLP course, especially scikit-learn. The project deliberately favors a smaller and fully implemented solution over a more complex neural model.

The primary research question is: How effectively can TF-IDF features combined with Logistic Regression identify six categories of toxicity in English online comments?

A secondary question is: What kinds of errors does the model make, particularly for rare toxicity labels and comments containing negation?

2. Dataset

The project uses the Jigsaw Toxic Comment Classification Challenge dataset hosted on Kaggle. According to the competition description, the training data consist of Wikipedia comments labeled by human raters for six forms of toxic behavior. The project uses the provided train.csv file and treats each label as an independent binary target [1].

The loaded file contains 159,571 comments and eight columns: an identifier, comment_text, and the six binary target columns. Missing comment text is replaced with an empty string before feature extraction.

Label	Positive examples	Share of comments
toxic	15,294	9.58%
severe_toxic	1,595	1.00%
obscene	8,449	5.29%
threat	478	0.30%
insult	7,877	4.94%
identity_hate	1,405	0.88%

Table 1. Positive label counts in the complete dataset.

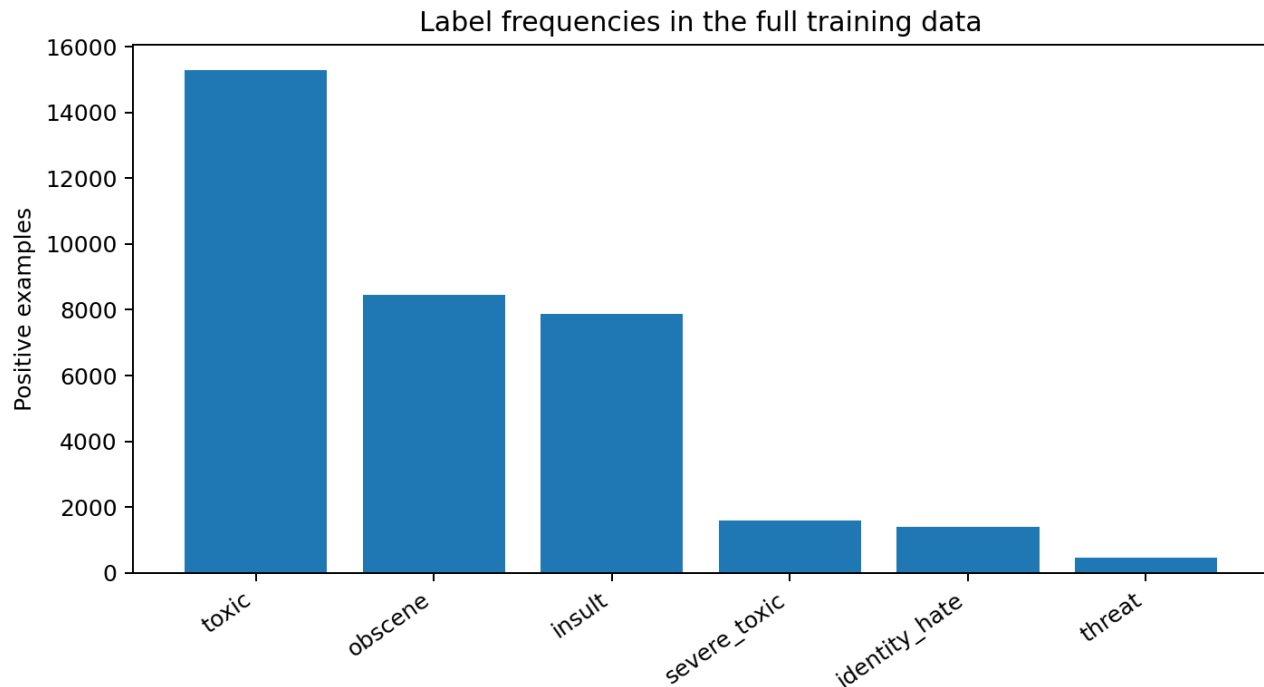


Figure 1. The target labels are strongly imbalanced; threat and identity_hate are much rarer than toxic, obscene, and insult.

3. Implementation and Pipeline

3.1 Data loading and target selection

The notebook searches the configured Jigsaw data folder recursively for train.csv, loads the file with pandas, selects comment_text as the input variable X, and selects the six target columns as y. The labels are stored explicitly in a LABELS list, which makes the target order visible and prevents accidental dependence on DataFrame column positions.

3.2 Train, validation, and test split

The project uses scikit-learn's train_test_split twice to create approximately 70% training data, 20% validation data, and 10% test data. random_state=42 is used in both splits so that repeated runs produce the same subsets. The final sizes are 111,699 training comments, 31,914 validation comments, and 15,958 test comments.

The training set is used to learn the text representation and classifier parameters. The validation set is used to inspect model behavior before the final test evaluation. The test set is held out until the model and preprocessing choices are fixed.

3.3 TF-IDF representation

Raw text cannot be passed directly to Logistic Regression, so comments are converted into sparse numerical vectors using scikit-learn's TfidfVectorizer [2]. The vectorizer is configured with lowercase=True, max_features=50000, ngram_range=(1, 2), min_df=2, max_df=0.95, and sublinear_tf=True.

The ngram_range setting includes both unigrams and bigrams. For example, the sentence "you are not stupid" can generate single-word features such as stupid and two-word features such as are not and not stupid. This provides more local context than unigram-only features, although it does not create a full semantic representation of sentence meaning.

To avoid data leakage, fit_transform is applied only to the training comments. The fitted vectorizer is then used with transform on validation and test comments. All three resulting matrices contain 50,000 features.

3.4 Multi-label classification

The classifier is Logistic Regression with the liblinear solver, max_iter=1000, and random_state=42. Logistic Regression is a regularized probabilistic classifier that can efficiently handle sparse input matrices [3].

Because each comment can have several toxicity labels at the same time, the model is wrapped in OneVsRestClassifier. This strategy trains one binary classifier per target label [4].

The six classifiers therefore answer separate questions such as "toxic or not toxic" and "insult or not insult". Multiple labels can be predicted as positive for the same comment. A probability threshold of 0.5 is used by the custom scoring function when displaying True/False decisions.

3.5 Persistence and interactive use

After training, joblib is used to save both the classifier and the fitted TF-IDF vectorizer. Saving both objects is necessary because new comments must be transformed with exactly the same vocabulary and IDF weights used during training. An optional Gradio interface calls the score_comment function and displays a probability and Boolean decision for each label.

4. Quantitative Evaluation

The evaluation uses precision, recall, F1-score, subset accuracy, and Hamming loss. Precision measures how often positive predictions are correct; recall measures how many true positive labels are recovered; F1 balances precision and recall. For multi-label data, micro averages aggregate decisions across all labels, while macro F1 gives each label equal weight. Subset accuracy is stricter because all six labels for a comment must match exactly. Hamming loss measures the fraction of individual label decisions that are wrong.

4.1 Validation results

On the validation split, the model obtains micro precision 0.8965, micro recall 0.5106, and micro F1 0.6506. The validation and test scores are very similar, which suggests that the test results are not an isolated outcome of one split.

4.2 Final test results

Label	Precision	Recall	F1	Support
-------	-----------	--------	----	---------

toxic	0.93	0.58	0.71	1480
severe_toxic	0.59	0.23	0.33	148
obscene	0.94	0.56	0.71	836
threat	1.00	0.11	0.20	37
insult	0.85	0.48	0.62	791
identity_hate	0.81	0.09	0.16	147

Table 2. Per-label results on the held-out test split.

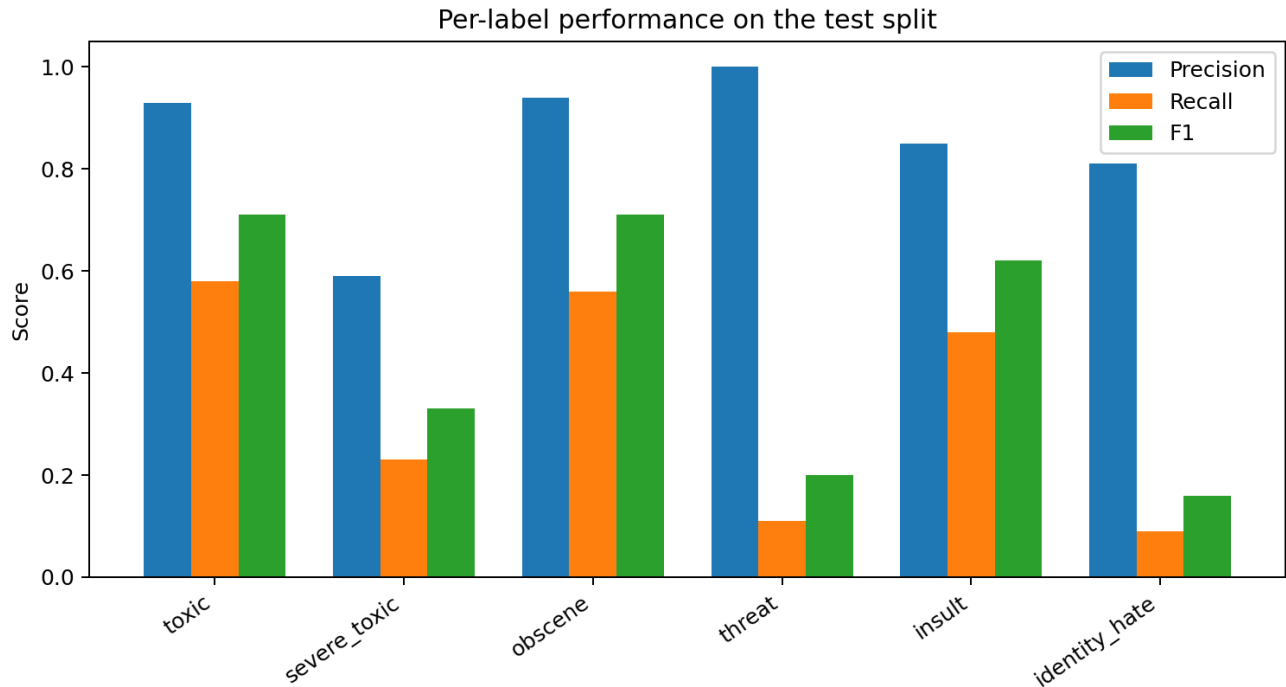


Figure 2. Precision is high across labels, while recall is much lower, especially for rare categories.

Overall test metrics: micro precision = 0.9026; micro recall = 0.5118; micro F1 = 0.6532; macro F1 = 0.4530; subset accuracy = 0.9211; Hamming loss = 0.0195.

4.3 Interpretation of the quantitative results

The most important pattern is the gap between precision and recall. Micro precision is about 0.90, while micro recall is about 0.51. Therefore, positive predictions are usually reliable, but the classifier misses a substantial number of true toxic labels. The fixed 0.5 decision threshold contributes to this trade-off: lowering the threshold could increase recall but would usually reduce precision.

Performance also differs considerably by label. The common categories toxic, obscene, and insult achieve F1 scores of 0.71, 0.71, and 0.62. The rarer categories severe_toxic, threat, and identity_hate achieve much lower F1 scores of 0.33, 0.20, and 0.16. This pattern is consistent with the strong class imbalance shown in Figure 1. For example, threat has only 478 positive examples in the complete dataset and only 37 positives in the test split, which makes it difficult to learn a robust decision boundary.

The subset accuracy of 0.9211 should not be interpreted by itself as evidence of uniformly strong classification. Most label values in this dataset are negative, so an accuracy-based metric can remain high even when recall for positive labels is low. Precision, recall, F1, and per-label results provide a more informative view of performance.

5. Qualitative Analysis

A small set of manually written comments was used to inspect what the classifier learned. The examples include clearly positive language, direct insults, and negated insults. The `score_comment` function transforms each new string with the fitted vectorizer and returns a probability for all six labels.

Comment	Toxic probability	Insult probability	Observation
I love you	0.0874	0.0441	Non-toxic
you are stupid	0.9981	0.9830	Strong toxic/insult prediction
you are not stupid	0.9996	0.9957	Negation does not reverse prediction
not stupid	0.9998	0.9917	Strong false positive under literal reading
you are fat	0.9859	0.9861	Strong toxic/insult prediction
you are not fat	0.9325	0.9236	Lower scores, but still toxic

Table 4. Selected manual predictions from the final notebook.

5.1 Negation as a failure case

The clearest qualitative weakness is negation. The classifier assigns `toxic=0.9981` to "you are stupid", but `toxic=0.9996` to "you are not stupid" and `toxic=0.9998` to "not stupid". A human reader can interpret the negated versions as denying the insult, but the model remains dominated by strongly toxic lexical features such as stupid. Similarly, "you are not fat" remains above the toxicity and insult thresholds even though its toxic probability is lower than for "you are fat".

This behavior is understandable for a TF-IDF model. Bigrams give the classifier access to local patterns such as not stupid, but the representation remains a weighted bag of lexical features. It does not construct a compositional semantic representation in which negation systematically reverses the meaning of the following adjective. The example therefore demonstrates a genuine limitation of the chosen representation rather than an implementation bug.

6. Challenges, Limitations, and Open Issues

- Class imbalance: rare categories have very few positive examples and correspondingly weak recall and F1 scores.
- Fixed threshold: all labels currently use a 0.5 probability threshold. Per-label threshold tuning on the validation set could improve the precision-recall balance.
- Negation and context: TF-IDF with unigrams and bigrams captures local lexical patterns but not deeper semantic composition, discourse context, irony, or pragmatic intent.
- Random multi-label splitting: the split is reproducible, but it is not explicitly stratified by the six-label combinations. Rare labels could therefore have somewhat different proportions across subsets.
- Portable data path: the final notebook uses a relative data path and the repository includes `data/README.md` with instructions for downloading and placing the Jigsaw dataset. This allows the same code to run on another computer without editing a personal Windows path.

- Evaluation scope: the current project reports several appropriate multi-label metrics, but additional analyses such as ROC-AUC, precision-recall curves, or threshold optimization could provide further detail.

7. Reproducibility and Repository Organization

In the final Git repository, `toxicity_sklearn.ipynb` is the main documented analysis entrypoint. The repository also includes `README.md` with installation and execution instructions, `requirements.txt` with package dependencies, `data_loader.py` for loading and filtering the data, and `data/README.md` with the official dataset source and expected directory structure. The dataset itself is not duplicated in the repository.

A recommended minimal repository structure is:

```
toxicity-detection/
  README.md
  requirements.txt
  toxicity_sklearn.ipynb
  data_loader.py
  models/
  data/README.md
  report/report.pdf
```

The Gradio cell is included as an optional final interface, and the notebook is structured to run from top to bottom after the dataset has been placed according to the README instructions.

8. Conclusion

This project demonstrates a complete and computationally efficient toxic-comment classification pipeline using techniques appropriate for the course. TF-IDF converts comments into sparse lexical features, while One-vs-Rest Logistic Regression provides six independent toxicity predictions. The final model achieves high micro precision but moderate recall, and its performance is substantially weaker on rare classes. The qualitative analysis further shows that lexical features alone are not sufficient for robust handling of negation. These strengths and weaknesses make the system a useful baseline: it is fast, reproducible, understandable, and easy to analyze, while still leaving clear directions for improvement through class balancing, threshold tuning, richer linguistic preprocessing, or more context-sensitive representations.

References

- [1] cjadams, J. Sorensen, J. Elliott, L. Dixon, M. McDonald, nithum, and W. Cukierski. Toxic Comment Classification Challenge. Kaggle, 2017. <https://www.kaggle.com/competitions/jigsaw-toxic-comment-classification-challenge/>
- [2] scikit-learn developers. TfidfVectorizer documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [3] scikit-learn developers. LogisticRegression documentation. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [4] scikit-learn developers. OneVsRestClassifier documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsRestClassifier.html>

[5] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011. <https://www.jmlr.org/papers/v12/pedregosa11a.html>